

Nama : Yehezkiel Hatoguan Siahaan
NIM : 245150600111012
Kelas : PAPB PTI-A

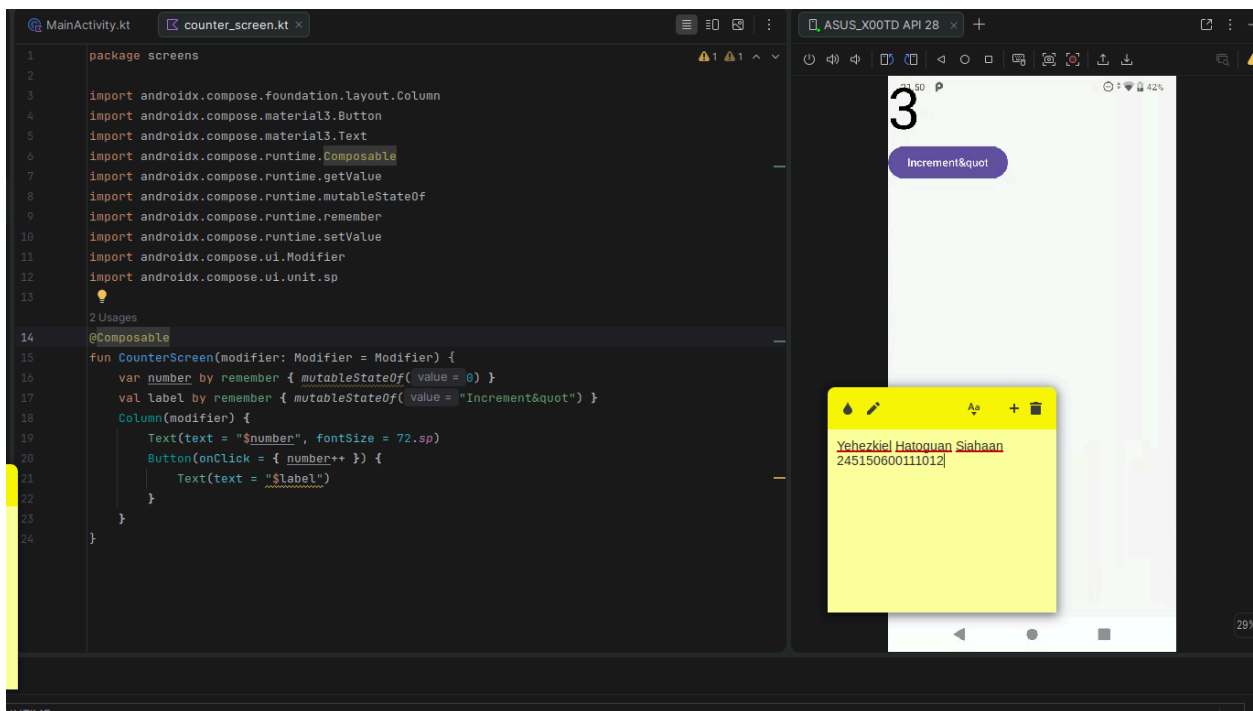
4.5.1 State Hoisting

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment"; beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.

- Apa yang terjadi setiap kali tombol "Increment" diklik?
- Apa yang menyebabkan hal tersebut terjadi?
- Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?
- Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

=



- Setiap tombol "increment" diklik, terjadi 2 proses utama, Perubahan state dimana `onClick = { number++ }` dieksekusi dan nilai bertambah 1. Tampilan UI tersebut otomatis diperbarui tanpa perlu mereload seluruh halaman
- Terjadi karena variabel `number` dibungkus menggunakan `mutableStateOf`. Ketika lambda `onClick = { number++ }` berjalan dan mengubah nilai `number`, Jetpack Compose

mendeteksi perubahan state dan langsung memicu proses Recomposition khusus pada elemen yang menggunakan variabel number.

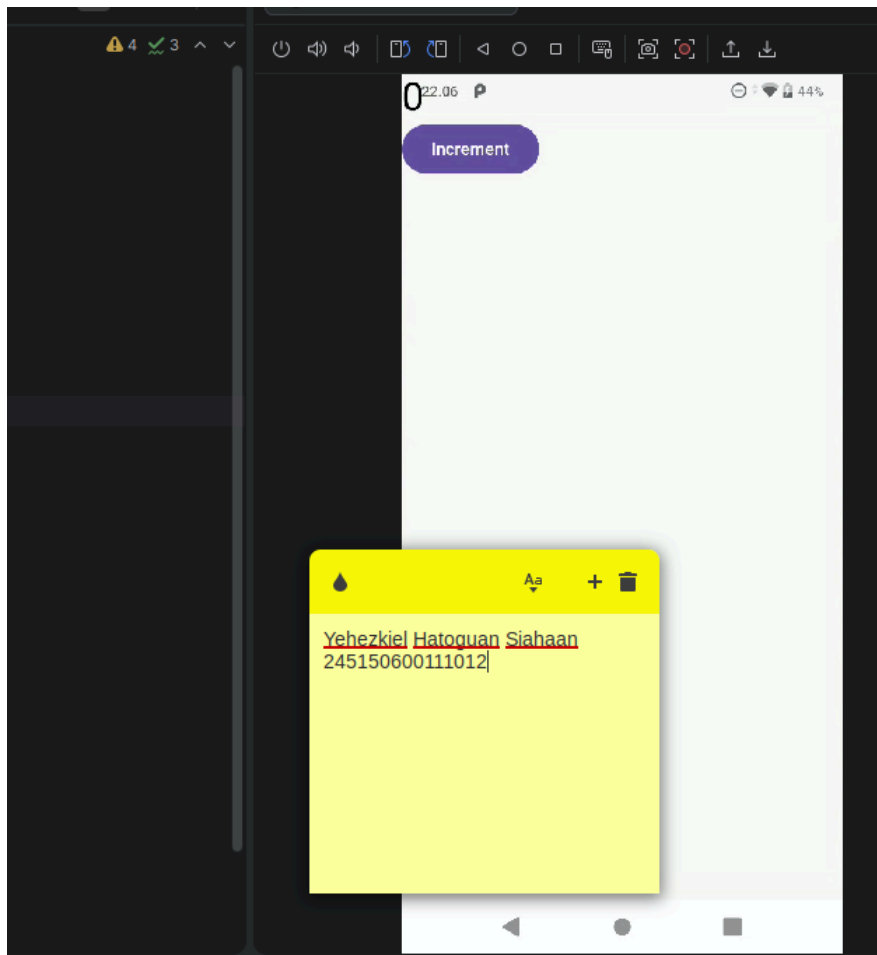
- c. Variabel state number (`var number by remember { mutableStateOf(0) }`)
- d. Variabel state label (`var label by remember { mutableStateOf("Increment") }`)

5. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

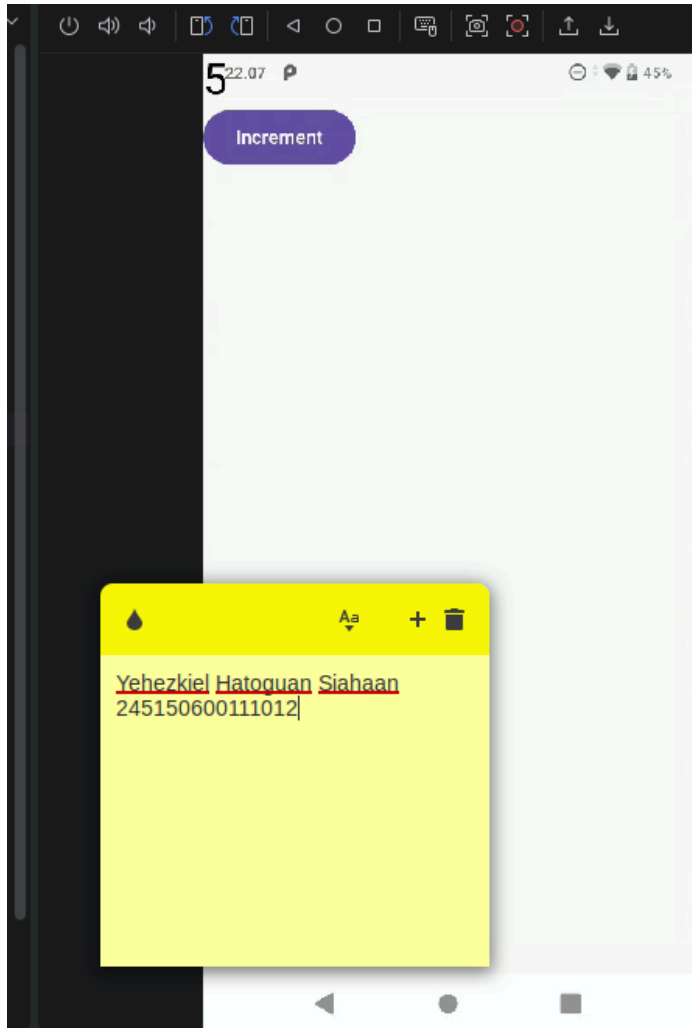
- a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?
- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

=

- a. Wge



- b. Sebelum
Sesudah



6. Langkah selanjutnya adalah melakukan state dan event hoisting pada state dan event yang sebelumnya terdapat di dalam Composable CounterScreen.

Ubah signature kode program pada bagian fungsi Composable CounterScreen dari yang sebelumnya:

```
fun CounterScreen(modifier: Modifier)
```

menjadi:

```
fun CounterScreen(modifier: Modifier,  
number: Int, label: String, onClick: () -> Unit)
```

Ubah body di dalam Composable CounterScreen menjadi:

```
// var number by remember { mutableStateOf(0) }
```

```
// val label by remember { mutableStateOf("Increment") }
```

```
Column(modifier) {
```

```
Text(text = "$number", fontSize = 72.sp)
```

```
Button(onClick = onClick) {
```

```
Text(text = "$label")
```

```
}
```

```
}
```

Ubah pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() dari yang sebelumnya:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

menjadi:

```
var number by remember { mutableStateOf(0) }
```

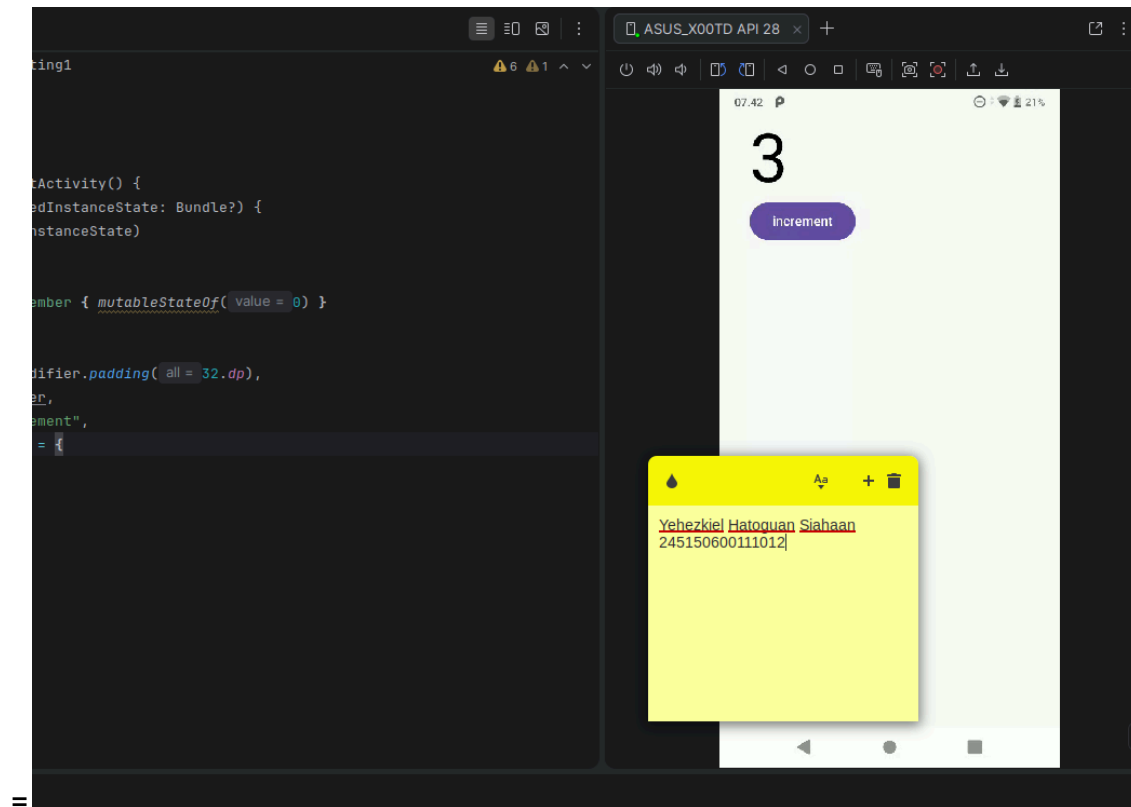
```
CounterScreen(Modifier.padding(32.dp),
```

```
number = number,
```

```
label = &quot;Increment&quot;,
```

```
onButtonClick = { number++ }  
)
```

Perhatikan bahwa deklarasi variabel state number dari yang sebelumnya dilakukan di dalam fungsi Composable CounterScreen, sekarang dideklarasikan di dalam fungsi onCreate() sebelum fungsi Composable CounterScreen dipanggil dari dalam fungsi onCreate().



7. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

- Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?
- Di bagian kode program manakah saat ini proses increment dilakukan?
- Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

=

- a. Program berjalan dan menampilkan hasil sama persis seperti tahap sebelumnya
- b. Di bagian `CounterScreen()`

```
c.     modifier = Modifier.padding(32.dp),
d.     number = number,
e.     label = "increment",
f.     onClick = {
g.         number++
h.     }
```

Dengan dilakukan di dalam lambda callback `onClick`, pada baris `number++`.

```
c. Column(modifier) {
    Text(text = "$number", fontSize = 72.sp)
    Button(onClick = onClick) { // Menggunakan callback dari parameter
        Text(text = "$label") // Menggunakan label dari parameter
    }
}
```

Di dalam `CounterScreen()`

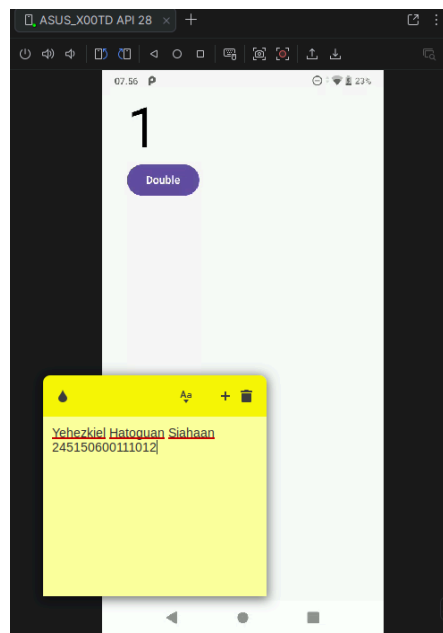
8. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna.

Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

- a. Label tombol "Increment" menjadi "Double";
- b. Nilai awal variabel `number` dari 0 menjadi 1.
- c. Ubah fungsi/lambda `onClick` pada pemanggilan `Composable CounterScreen` di dalam fungsi `onCreate()` sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.
- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi `Composable CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

=

D.



Kode label

```
label = "Double",
```

Kode variable awal menjadi 1

```
var number by remember { mutableStateOf(1) }
```

Kode menggandakan number

```
number*=2
```

9. Jalankan program,

a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

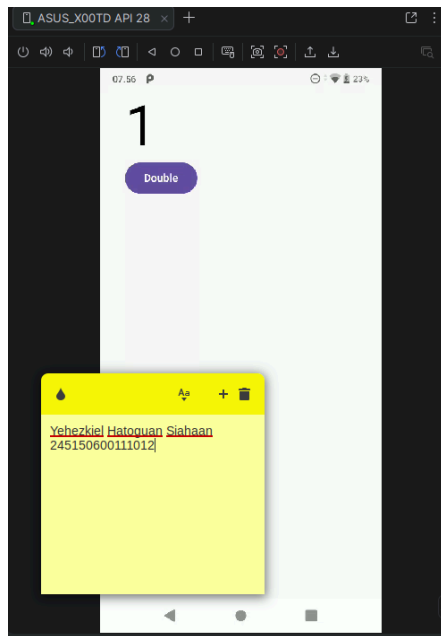
b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri?

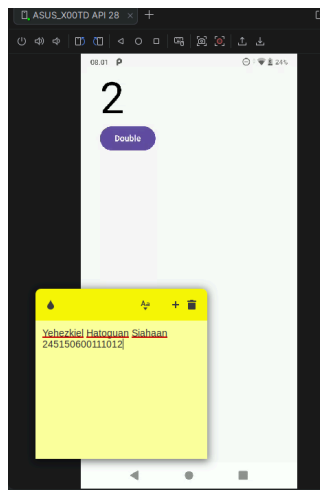
Jika ada sebutkan perubahan yang dilakukan.

=

a. Sebelum digandakan



Setelah digandakan (x 2)



- b. Didalam function onCreate mainactivity
- c. Tidak ada semua perubahan berada pada fun onCreate()

10. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

= Fleksibilitas dan Reuasability komponen menjadi stateles, pemisahaan tanggung jawab antara screen dan juga logika, serta maintainbality atau mudah dipelihara.

4.5.2 Perubahan Konfigurasi Pra-Percobaan

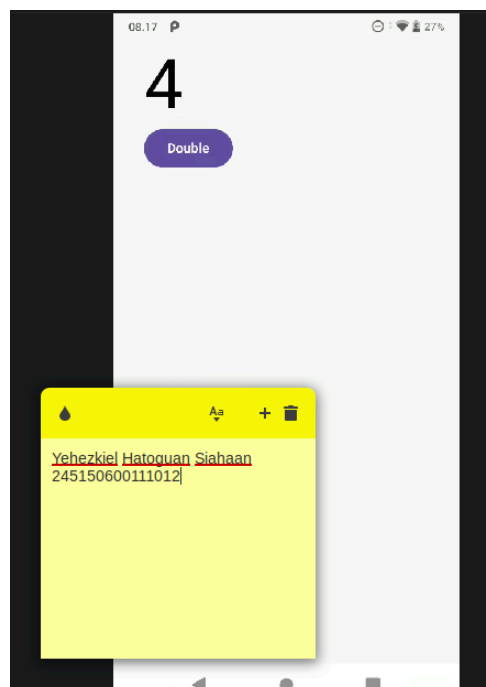
1. Untuk percobaan terkait Perubahan Konfigurasi aplikasi Android, gunakan Project Android Studio yang sama dengan kode program percobaan State Hoisting setelah menyelesaikan tahap nomor 6 percobaan tersebut.
2. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel number).

Aktivitas Percobaan

1. Jalankan program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya

=

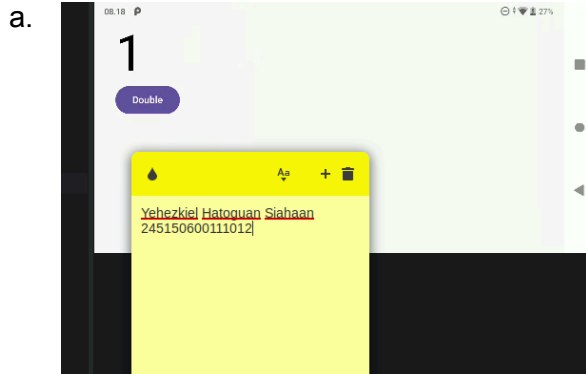


2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.

a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.

b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

=



b. Angka yang muncul kembali ke nilai awal, hal ini terjadi karena rotasi layar merupakan Configuration Change, Secara default, sistem android akan destroy dan membuat ulang komponen activity. Fungsi remember hanya mempertahankan nilai state saat Recomposition bukan saat Activity Recreation akibat rotasi layar.

Ubah bagian kode program di dalam fungsi onCreate() seperti berikut ini:

```
var number by remember { mutableStateOf(1) }
```

menjadi:

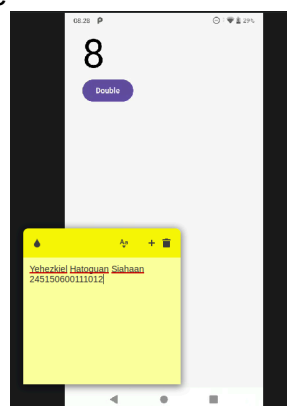
```
var number by rememberSaveable { mutableStateOf(1) }
```

Lakukan import dependency atas perubahan konstruk remember menjadi rememberSaveable yang dilakukan (dengan memposisikan cursor pada konstruk rememberSaveable tersebut dan melakukan refactoring dengan menekan kombinasi tombol Alt-Enter di jendela editor Android Studio).

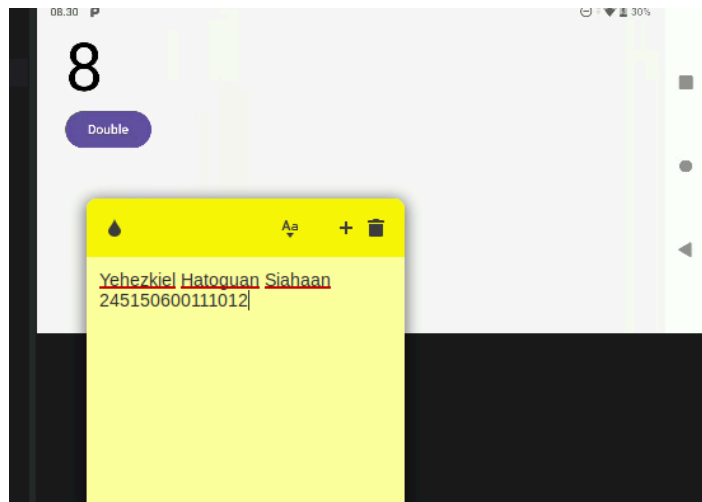
Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double"; sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

= Sebelum landscape



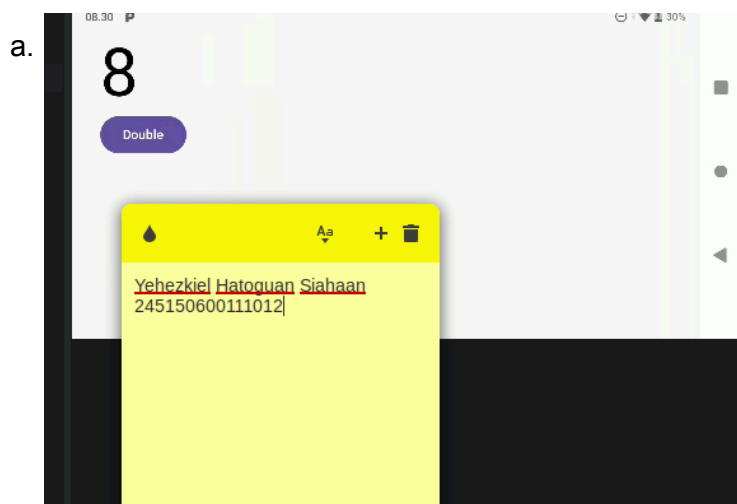
Setelah landscape



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.

- a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.
- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?
- c. Apa fungsi dari konstruk rememberSavable pada kode program Android yang membedakannya dari konstruk remember?

=



- b. Angka yang ditampilkan tetap 8 karena sudah menggunakan rememberSaveable
- c. rememberSaveable menyimpan memori perubahan activity sehingga activity di rotasi Compose mengambil data yang tersimpan dan memasukkannya ke variabel yang dimunculkan. Sementara remember hanya menitipkan nilai ke dalam Composition,

ketika Compose di rebuild (recomposition). Nilai lama dipanggil lagi. Namun, ketika Activity dihancurkan akibat rotate layar memori Composition ikut terhapus.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

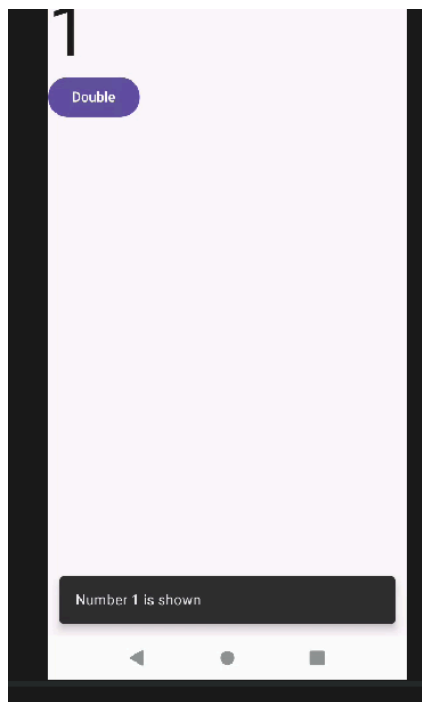
= Contoh kasus nyata yang tepat menggunakan `rememberSaveable` adalah pada formulir pendaftaran atau pencarian, seperti menyimpan teks yang sedang diketik pengguna di dalam `OutlinedTextField` atau status tombol *checkbox* persetujuan syarat dan ketentuan. Manfaat utama dari penerapannya adalah menjaga pengalaman pengguna (*User Experience*) tetap mulus karena data yang sudah diisi tidak akan terhapus secara tidak sengaja saat pengguna memutar layar HP (*portrait* ke *landscape*) atau ketika aplikasi berpindah ke latar belakang (*background*) akibat menerima panggilan telepon.

4.5.3 LaunchedEffect

Aktivitas Percobaan 1

1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double", apa yang ditampilkan di dalam layar?
 - b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?
 - c. Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana?
2. Klik button "Double" satu kali,
 - a. Apa yang terjadi?
 - b. Mengapa hal tersebut terjadi? Jelaskan!

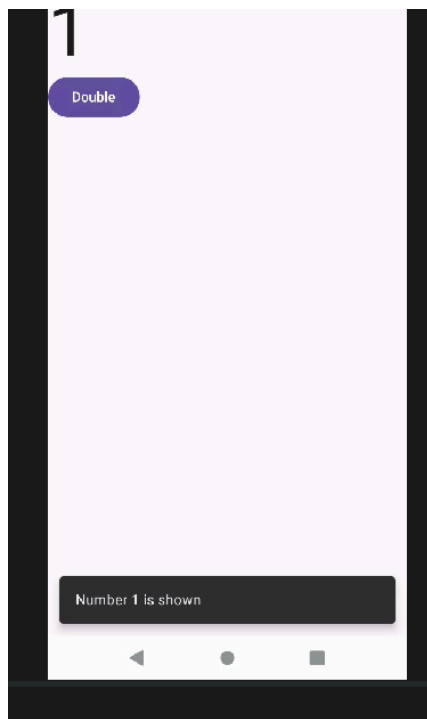
= a.



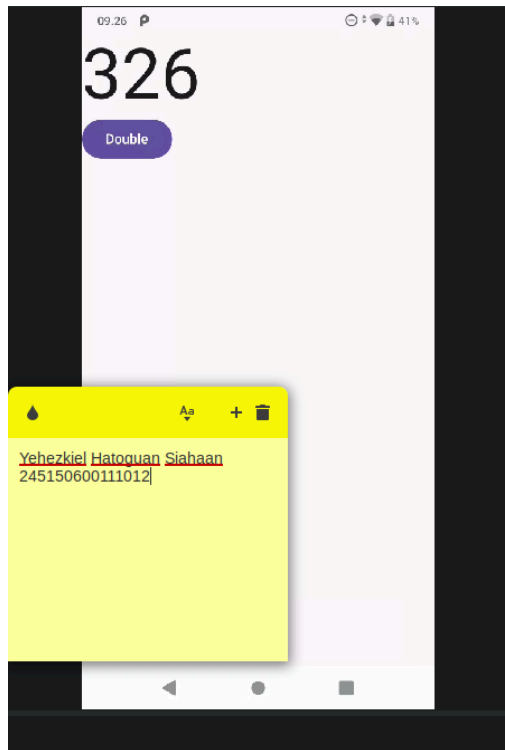
- b. Pesan *Snackbar* langsung muncul saat pertama kali aplikasi dijalankan karena `LaunchedEffect` selalu dieksekusi minimal satu kali pada saat pertengahan proses inisialisasi awal (*initial composition*).
- c. Pemicu (*Trigger*): Pesan dipicu oleh perubahan nilai (*state change*) pada variabel `number`. Setiap kali tombol diklik dan nilai `number` berubah, `Compose` mendeteksi bahwa *key* pada `LaunchedEffect(number)` telah berganti, lalu memicu ulang eksekusi blok kode di dalamnya.

Aktivitas Percobaan 2

1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?
 - b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?
 - c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana?
- =
- a. Sama seperti sebelumnya memunculkan `snackbar` `number is shown`



b. Yang tampil berupa angka random 326



c.

```
suspend fun getData() : Int {  
    delay(2000) // simulating network  
    return Random.nextInt(100, 1000)  
}
```

2. Dari kedua percobaan terkait dengan side-effect dengan LaunchedEffect,

a. Jelaskan apa perbedaan antara LaunchedEffect(state) dengan LaunchedEffect(Unit)?

b. Kapan operasi di dalam LaunchedEffect(state) dengan LaunchedEffect(Unit) dijalankan?

=

- a. -LaunchedEffect(state) (Dynamic Key): Akan memantau nilai variabel state yang dimasukkan sebagai parameternya. Setiap kali nilai state tersebut berubah/diperbarui, blok kode di dalam LaunchedEffect akan dibatalkan dan dijalankan ulang (*re-launched*).
-LaunchedEffect(Unit) (Constant/Static Key): Menggunakan Unit (konstanta tunggal di Kotlin) sebagai *key*. Karena nilai Unit tidak pernah berubah, blok kode di dalamnya hanya akan dieksekusi satu kali saja selama siklus hidup Composable tersebut di layar.

- b. -`LaunchedEffect(state)` dijalankan saat: Pertama kali komponen digambar di layar (*Initial Composition*).

Setiap kali terjadi perubahan nilai pada variabel state yang dipantau (misalnya ketika number bertambah dari 0 ke 1).

-`LaunchedEffect(Unit)` dijalankan saat:

Pertama kali komponen digambar di layar (*Initial Composition*).

TIDAK AKAN dijalankan ulang meskipun komponen mengalami *recomposition* atau ada *state* lain yang berubah di dalam layar.